

Identity Best Practices: Security Hardening

The practices that keep your Azure identity secure — least privilege, reviews, MFA, and proper delegation

 Intermediate

 15 Minutes

 AZ-104 · Module 01

Why This Matters

Bad identity practices are how breaches happen. Weak passwords, unused accounts, overly permissive roles, missing audit trails — these are the exploits hackers use. This doc covers the practices that keep your Azure identity secure: least privilege, regular reviews, MFA, and proper delegation.

Before You Start

Prerequisites	Entra ID Overview, RBAC Fundamentals, Access Control Scenarios
Time to understand	15 minutes
Difficulty	 Intermediate (security mindset helpful)
What you'll learn	How to secure identity and access in Azure

Best Practice 1: Least Privilege Access

The Principle

Least Privilege = Give people only the minimum access they need, nothing more.

Wrong (Excessive Access)

Employee: "I need to deploy code"
Manager: "Okay, I'll give you Owner role on the subscription"

Result: Employee can delete EVERYTHING, including databases ✗

Right (Least Privilege)

Employee: "I need to deploy code"

Manager: "Okay, I'll give you Container Registry Contributor (push images only)"

Result: Employee can only push to registry, can't delete other resources ✓

Implementation

Step 1: Determine minimum needed

"What is the smallest role that lets them do their job?"

Step 2: Start with Reader

If you don't know, start with read-only, expand only if needed

Step 3: Remove over time

At end of project: "Do you still need Contributor?" → Remove if not ✓

Step 4: Review quarterly

Question every role: "Is this still necessary?"

Roles Ranked by Privilege (Most to Least)

1. Owner (everything, delete RGs)

└ Give to: IT directors, cloud architects only

2. Contributor (create/modify, NOT delete RGs)

└ Give to: Senior engineers, ops teams

3. Operator roles (specific operations like "start VMs")

└ Give to: Support staff, automation accounts

4. Reader (read-only, no changes)

└ Give to: Auditors, consultants, everyone else by default

5. Custom roles (exact permissions needed)

└ Use when no built-in role fits

Best Practice 2: Regular Access Reviews

The Problem

Access Creep (Permissions grow over time):

```
Year 1: Employee hired → assigned Contributor role
Year 2: Moves to different project → keeps old Contributor role
Year 3: Gets promoted → gets new role, doesn't lose old ones
Year 4: Employee leaves → access not revoked for 6 months
Result: Former employee still has access X
```

Implementation

Quarterly Access Review Checklist:

- └ Question 1: "Are they still in the role's group?"
- └ Question 2: "Does the job still require this role?"
- └ Question 3: "When was this role last used?"
- └ Question 4: "Has their responsibility changed?"
- └ Question 5: "Any unusual activity from this account?"

For YES answers → Keep

For NO answers → Remove (audit the removal)

Azure Policy to Help

Policy: "Inactive accounts get read-only access after 90 days"

Logic:

- └ If no sign-in in 90 days
- └ Remove Contributor role
- └ Keep Reader role (if absolutely needed)
- └ Send email asking "Can we disable this account?"

Result:

- └ Inactive accounts can't cause damage
- └ Budget impact reduced
- └ Security surface shrinks

Best Practice 3: Enable Multi-Factor Authentication (MFA)

The Problem

Weak Passwords Get Hacked:

```
Attacker gets password: P@ssw0rd123
├ Without MFA: Logs in immediately ✗
└ With MFA: Can't log in (no phone/app) ✓
```

Implementation

Option 1: Require for Admin Roles Only

- └ Conditional Access Policy
 - └ If role is Global Admin → Require MFA
 - └ Balance: Moderate security + usability

Option 2: Require for All Users

- └ All users must register MFA method
- └ Highest security, impacts user experience

Option 3: Risk-Based (Conditional Access)

- └ Require MFA if:
 - └ Sign-in from unusual location ✓
 - └ Sign-in from unfamiliar device ✓
 - └ Multiple failed login attempts ✓
 - └ Normal sign-in from known location → No MFA needed ✓
- └ Best balance of security + usability

MFA Methods (Order by Security)

1. Windows Hello / Biometric
 - └ Most secure, works only on configured device
2. Microsoft Authenticator app
 - └ Very secure, phone required
3. FIDO2 security key
 - └ High security, requires special hardware
4. SMS/Text message
 - └ Least secure (vulnerable to sim-jacking)
 - └ Better than nothing, but don't use alone

Recommendation

Require Authenticator app or Windows Hello for admins.

Best Practice 4: Monitor Sign-in Activity

The Problem

Suspicious Activity = Breach in Progress:

Attacker steals password

- └ Signs in from China (your company is in US)
- └ Accesses data
- └ Deletes resources
- └ You don't notice for weeks X

Implementation

Entra ID → Monitoring → Sign-in logs

What to Look For:

- └ Sign-in from unusual country
- └ Sign-in from unknown device
- └ Multiple failed sign-in attempts (password guessing)
- └ Sign-in outside business hours

- └ Bulk changes (deleting users, roles) by one person
- └ Access to sensitive data by unusual users

Set Up Alerts

Alert 1: Failed sign-in attempts

- └ If > 5 failed attempts in 5 minutes
- └ Email IT immediately
- └ Lock account for 15 minutes (Entra feature)

Alert 2: Sign-in from unusual location

- └ If user signed in from new country
- └ Require MFA re-verification
- └ Send notification to user

Alert 3: Unusual resource access

- └ If deleted users / roles changed by unusual person
- └ Flag for immediate investigation
- └ Require approval for such operations

Best Practice 5: Use Groups, Not Individual Assignments

The Problem

Individual Assignments Don't Scale:

- Assign role to 50 individuals
- └ "sara@company.com" → Contributor
- └ "mike@company.com" → Contributor
- └ ... (48 more times)

- └ Someone leaves → Did you remove all 12 of their roles?
- └ New person joins → Did you add them to all required roles? X

Implementation

Use groups instead:

- └ Create grp-engineering-contributors
- └ Add 50 people to the group
- └ Assign Contributor to the GROUP (once)
- └ Someone leaves → remove from group, they lose all roles ✓
- └ New person → add to group, they get all roles ✓

Policy: "All role assignments must be to groups, not individuals"

- └ Exception: Service principals can have individual roles
- └ Result: All human access is via groups, trackable, scalable

Best Practice 6: Separate Duty / Approval Required

The Problem

One Person Can Approve Themselves:

Alice wants Contributor role on Production

- └ Alice submits request
- └ Alice's manager approves (she IS the manager!)
- └ Alice approves her own request
- └ No one else reviews → potential security risk X

Implementation: Approval Chain

- └ Request: Employee submits (I need role X on resource Y)
- └ Approval 1: Manager (does this make sense for their job?)
- └ Approval 2: Security team (is the resource sensitive?)
- └ Approval 3: Resource owner (do you want this person?)
- └ Execution: IT provisions role only after all approvals ✓

Exception: Same person cannot approve their own request

Azure Entitlement Management

Azure AD → Identity Governance → Entitlement Management

Setup:

- └ Create access package (e.g., "Production-Contributor")
- └ Define who can request (all employees)
- └ Define approval workflow (manager + security)
- └ Set expiration (roles auto-expire after 90 days)
- └ Auto-review (request approval before expiration)

Benefit:

- └ Formal audit trail ✓
- └ Proper separation of duty ✓
- └ Auto-cleanup (no orphaned roles) ✓
- └ Full compliance evidence ✓

Best Practice 7: Disable Unused Accounts Quickly

The Problem

Orphaned Accounts = Security Risk:

Employee leaves on Friday

- └ Managers forget to disable account
- └ Account still active Monday
- └ Credentials stolen/sold on dark web Tuesday
- └ Attacker has access Wednesday (nobody notices yet)
- └ Breach discovered 6 months later X

Implementation

Offboarding Process

- └ Step 1: Day of departure → Disable account in Entra ID
- └ Step 2: Revoke all role assignments
- └ Step 3: Remove from all groups
- └ Step 4: If not needed in 30 days → Delete account
- └ Step 5: Audit: Search for old accounts, disable any orphaned ones

Automation & Verification

Automation:

- └ HR system → Email IT when employee terminates
- └ IT runs script to:
 - | └ Disable account
 - | └ Remove from groups
 - | └ Send notification
- └ Manager confirms cleanup complete

Verification:

- └ Quarterly: Check for disabled accounts >90 days old, delete them

Best Practice 8: Use Service Principals with Passwords Only When Necessary

The Problem

Service Principals with Passwords = Shared Secrets:

App uses hardcoded password

- └ Password stored in code (bad!)
- └ Password stored in config files (bad!)
- └ Password rotated slowly/never (bad!)
- └ Password exposure = app compromise X

Implementation

Bad: Service principal with password

app → connects to Azure → authenticates with password

- └ Password shared everywhere

Good: Service principal with certificate

app → connects to Azure → authenticates with certificate

- └ Certificate renewed automatically, harder to steal

Best: Managed Identity

app → runs on Azure → auto-authenticated (no credentials needed!)

- └ Azure manages all credential rotation internally ✓

When Each Applies

Managed Identity (PREFERRED)

- └ Use when: App runs on Azure VM, App Service, Container, Function

- └ Benefit: Zero secrets to manage ✓

Service Principal + Certificate

- └ Use when: App runs outside Azure, certificate can be renewed

- └ Benefit: More secure than passwords

Service Principal + Password (AVOID)

- └ Use only if: Certificate isn't possible (rare)

- └ Caveat: Must rotate password every 90 days minimum

Best Practice Checklist

☐ Implement Least Privilege

- └ Audit: Are your lowest roles even lower than they need to be?

☐ Quarterly Access Reviews

- └ Schedule: First Monday of each quarter
- └ Owner: IT security team

❑ Require MFA for Admins

- └ Minimum: All roles with "Admin" in the name
- └ Better: Risk-based MFA for all users

❑ Monitor Sign-in Logs

- └ Tool: Entra Sign-in logs
- └ Alert: Unusual geography, failed attempts, bulk changes

❑ Use Groups Exclusively

- └ Audit: Do any role assignments exist to individual users?
- └ Policy: Enforce group-based assignments

❑ Require Approval for Sensitive Roles

- └ Setup: Entitlement Management with approval workflow
- └ Sensitive roles: Owner, User Administrator, Global Admin

❑ Offboard Quickly

- └ SLA: Disable account within 24 hours of departure
- └ Verify: Monthly audit of disabled accounts

❑ Use Managed Identities

- └ Apps on Azure → Managed Identity
- └ Apps off Azure → Certificate or (rarely) password

How This Connects to Other Topics

Related to All Modules

- **Every Azure operation** goes through identity
- **Policies enforce best practices** (MFA, encryption, etc.)
- **Audit logs** prove compliance

Related Labs

- **Lab 02: RBAC & Azure Policy**
- Lab exercises: Set up MFA, review access, configure alerts

See It In Action

Related labs:

- **Lab 02: RBAC & Azure Policy**
- Lab exercises: Set up MFA, review access, configure alerts

Key Takeaways

Summary

- **Least privilege** = minimal access needed, nothing more
- **Regular reviews** = catch permission creep before it becomes a breach
- **MFA** = makes password theft much less valuable
- **Groups** = easier to manage than individual assignments
- **Audit logs** = prove who did what, when, for compliance
- **Approval workflows** = prevent unilateral access grants
- **Quick offboarding** = disable accounts before they become liabilities

Next Steps

1. **Review:** Read this doc (you're here)
2. **Implement:** Start with MFA and quarterly reviews
3. **Monitor:** Set up sign-in alerts
4. **Audit:** Migrate individual assignments to groups

Azure AZ-104	Module	Generated
Professional Concept Documentation	01 — Identity & Governance	June 15, 2026

© 2026 — AZ-104 Learning Hub. For study purposes only.